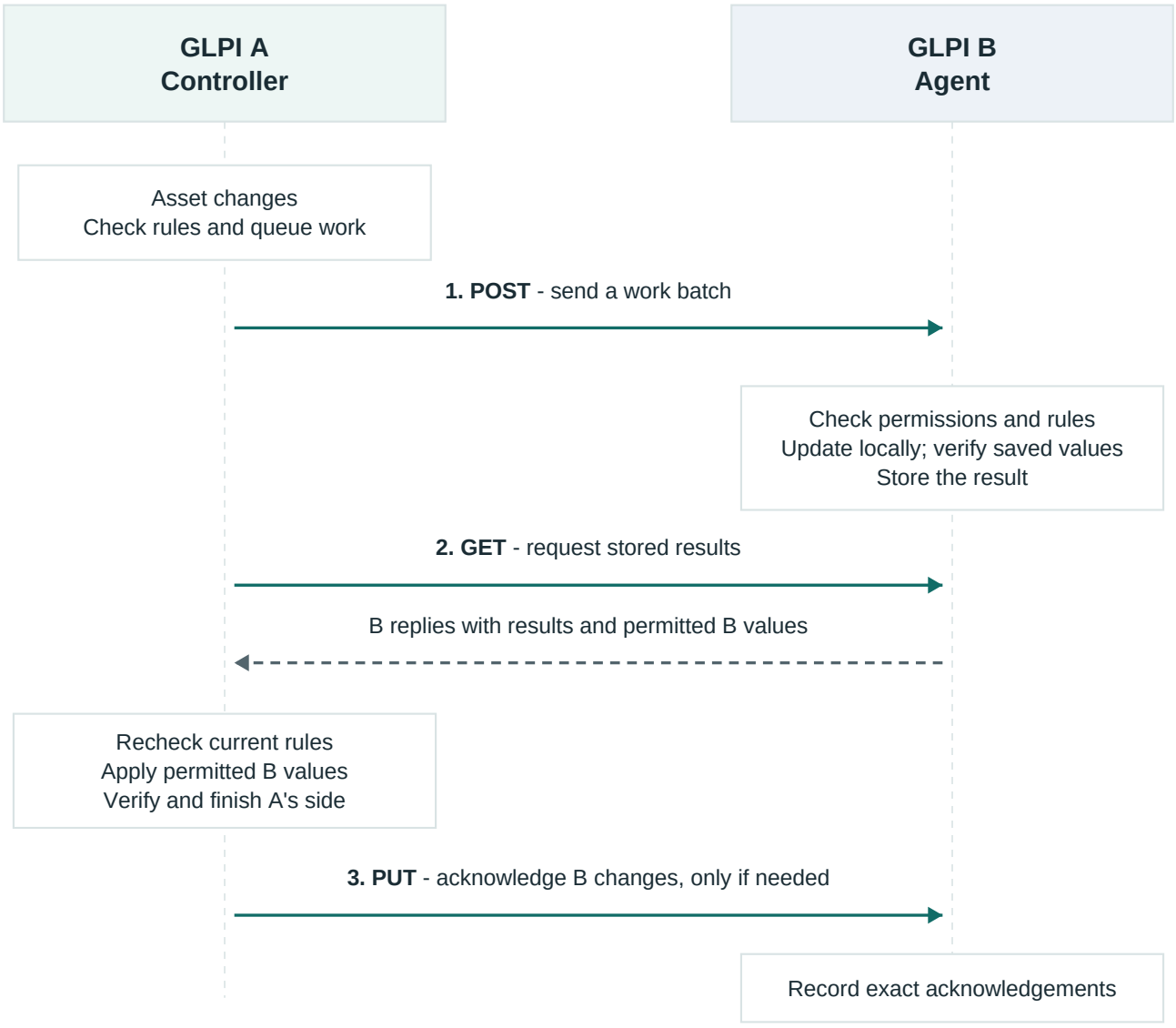
How AssetSync Works

Design versus implementation. This guide describes the complete diagram-based design being built. The new execution path currently supports only a limited part of it. Page 11 explains what is verified and what remains unfinished.



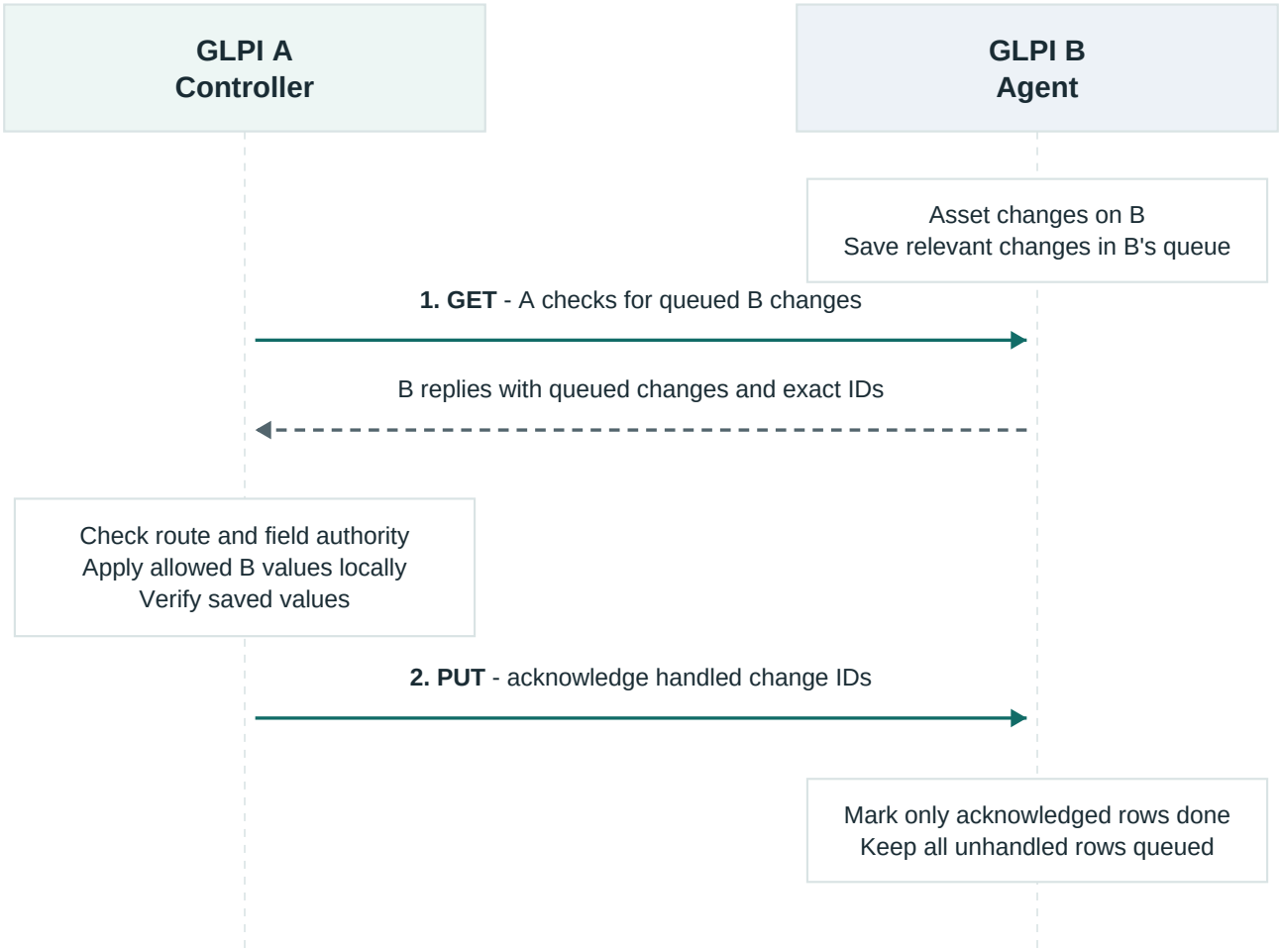
Read from top to bottom. Solid arrows are requests from A. The dashed arrow is B's reply to the GET, not another request. All communication uses authenticated HTTPS.

Normal successful batch: two work requests, POST and GET. A third request is needed only to acknowledge returned B change records. Authentication, settings updates, extra polling and retries are counted separately.

For edits made only on B, A periodically checks B's saved change queue and acknowledges handled records. B does not need a separate callback to A in this design. The next page shows this B-started flow.

Changes Starting in GLPI B

The edit starts on B, but A still makes the HTTPS requests. This normal path applies when the field rules allow A to accept B's value.



Read from top to bottom. Solid arrows are requests from A; the dashed arrow is B's response. A checks periodically, even when nothing changed on A.

Normal successful batch: two requests. GET the changes, then PUT the exact acknowledged IDs. A acknowledges only changes it successfully handled. Failed or conflicted changes stay queued for review or retry.

If B needs a correction: after the first GET, A sends a POST containing the correction and covered change IDs, then GETs the verified result. That is three work requests in total. Covered IDs are settled only after successful handling; they do not need a separate PUT.

Authentication, settings updates, extra polling and retries are additional requests. B keeps durable change records until the appropriate acknowledgement or successful correction settles them.

Setup and Field Mapping

1. The two sides

GLPI A is the controller. This is your central installation. It owns the connections, entity routes, field mappings, source-of-truth rules and billing calculations.

GLPI B is the destination or client installation. Its Agent receives work, checks local permissions, reads or updates assets using GLPI's own methods, and saves the results. A owns the business rules, but B can reject work its service account is not allowed to perform.

2. You configure what should sync

On A, you configure a connection to each B installation, then define an **Entity Route**. For example: A's **My Organization > Client One** entity routes to B's **Client One** entity, with Computers and Monitors selected as the asset types.

Only assets matching an enabled route and asset type are eligible. One A asset can participate in routes to different B connections. Each connection has its own asset link and sync progress.

3. You map the fields

Field mappings describe where information goes. The overall requirement includes **native-to-native, native-to-custom, custom-to-native and custom-to-custom** mappings. Native fields are built into GLPI; custom fields are added through the Fields plugin.

For example, A's native Name could feed both B's native Name and B's custom Asset Name. A dropdown mapped to text should send its readable label, not its database ID. Dropdown-to-dropdown needs an independent destination lookup because A and B can use different IDs for the same option.

Each combination needs explicit implementation support. Appearing in GLPI's search options does not automatically make a field safely writable.

From an Edit to a Batch

4. You choose who controls each field

Each mapping has a **source of truth**: the side whose value should be followed.

CHOICE	MEANING
GLPI A	A's value controls that mapping.
GLPI B	B's value controls that mapping.
Both	The newer genuine field change wins, where reliable evidence exists.

Both must not simply compare whole-asset timestamps. Copying B's edit into A can make A look newer without representing a new human edit. Missing or ambiguous evidence must produce a visible conflict, not a guessed winner.

5. A detects relevant changes

When someone adds, edits or transfers an asset in A, the plugin checks whether it falls within an enabled route and whether relevant mapped information changed.

Eligible work is recorded in a **local queue**: a saved list of work waiting to be processed. The background worker handles the network exchange later, keeping slow remote operations out of the user's normal save request.

Unrelated edits should not produce asset-sync requests. Sync writes must also be distinguished from genuine new edits to prevent endless copying back and forth.

6. A prepares and sends a batch

A groups compatible queued work into a bounded batch. Each asset's work carries an identity, route, target entity, mapped values, change evidence and the settings version it follows.

A sends the batch to B over authenticated HTTPS. B keeps a versioned copy of A's relevant settings, updated when needed. Disabled connections and invalid or superseded settings must not silently continue writing.

> **Accepted does not mean synced.** Acceptance means B saved the work request. It does not prove that an asset was updated or that A has finished its part.

Source-of-Truth Examples

All three examples use the mapped **Comments** field on an existing confirmed asset pair. They illustrate the target rules, not additional verified product coverage. Assume permissions and all other safety checks pass.

GLPI A controls the field

STAGE	GLPI A	GLPI B
Before sync	Assigned to Finance	Assigned to Sales
After sync	Assigned to Finance	Assigned to Finance

Result: A stays unchanged; B receives A's value. Even if someone edited B more recently, B does not control this mapping.

GLPI B controls the field

STAGE	GLPI A	GLPI B
Before sync	Assigned to Finance	Assigned to Sales
After sync	Assigned to Sales	Assigned to Sales

Result: B stays unchanged; A receives B's value. Starting the sync from A does not reverse the selected source of truth.

Both: the newer genuine field edit wins

STAGE	GLPI A	GLPI B
Before sync	Assigned to Finance Edited at 09:00	Assigned to Sales Edited at 09:10
After sync	Assigned to Sales	Assigned to Sales

Result: B's later edit is copied to A. If A later has a genuine edit at 09:20, that edit can win instead. Times are on the same day in Brunei time, with trustworthy, comparable change evidence.

> **When Both cannot choose:** if the values differ but the change times are equal, missing or unreliable, keep both values unchanged and show a field conflict. If the values already match, no write is needed.

First-copy exception: a brand-new B asset initially receives enabled mapped A values once. These examples apply after the asset is linked and normal field authority takes over.

B Does the Local Work

7. B finds or creates the asset

A and B maintain a **confirmed link** identifying which records represent the same asset. Their numeric asset IDs do not need to match.

An existing link is checked before updating. Without one, matching must be complete and unambiguous. A failed lookup, denied permission or duplicate match must never be treated as permission to create another asset.

Under the diagram's first-copy rule, a genuinely new B asset receives all enabled mapped A values once, including B-authoritative mappings. After creation and confirmation, normal per-field authority applies.

8. B applies the field rules locally

For an existing asset, B copies A-authoritative values, returns B-authoritative values, and resolves supported Both mappings using trustworthy change evidence. **Unmapped fields remain unchanged.**

The Agent uses supported local GLPI and Fields-plugin methods under its configured service account. It checks permissions, field types, entity scope and current values. It then rereads the saved values to verify persistence.

This local update path is intended to produce normal History entries. A successful HTTP response alone is not enough evidence that a field changed.

9. A collects and finishes the work

A fetches B's stored results. Before making any A-side changes, it checks that the route, mappings and relevant source values are still valid.

A applies permitted returned B values locally and verifies them. Where configured, billing calculations then use the updated inputs. **Billing runs on A**; it is not a separate B billing engine.

B's completion and A's completion are separate stages. An asset is not fully synchronized merely because B finished its part.

> **The completion chain:** work accepted, local execution, persisted-value verification, result collection, and A-side finalization. Each stage must be reported honestly; none can be assumed from an earlier stage.

Example: A New Asset

You create a Computer named **DMOS-LT-001** directly in A's **Client One** entity. Its serial number is **SN-DEMO-001**. An enabled route maps that entity to **Client One** on B and allows Computers. Name and Comments are enabled mappings.

This illustrates the target first-copy behavior, not a completed live test. Assume the serial is unique, no B copy exists, and the service account has the required read, create and field permissions.

STAGE	GLPI A	GLPI B
Before creation	No asset yet.	No linked or matching asset.
After saving on A	DMOS-LT-001 Entity: Client One	Still no asset. Sync is queued, not yet complete.
After verified sync	DMOS-LT-001 Entity: Client One Original A record remains.	DMOS-LT-001 Entity: Client One B copy created and link confirmed.

What happens next

1. A checks eligibility. The saved entity and Computer type match the route, so A records the new work in its local queue.

2. A sends the work. Its worker sends the route, identity, mapped values and settings version to B in a batch.

3. B checks before creating. B checks links and permitted identifiers locally. Only a complete, unambiguous no-match result permits creation through GLPI's normal local add method. B saves the created ID, verifies the values and retains the result.

4. A confirms the result. A collects B's result and confirms the link under the current route and identity checks. Later syncs use that link rather than creating another copy.

> **First copy versus later updates.** For this new B asset, all enabled mapped A values are used once, even for B-authoritative fields. After confirmation, normal A, B or Both rules apply. Matching an existing B asset must not trigger this first-copy overwrite.

A denied or incomplete lookup is not proof of absence. Duplicate identifiers, permission errors and uncertain creation outcomes must remain visible; they must not trigger another blind create.

Example: Transfer to a Client

A Computer named **DMOS-LT-002**, serial **SN-DEMO-002**, starts in your **My Organization** entity on A. No enabled route matches it there. You later transfer the same A asset to **My Organization > Client One**.

> **Saved route:** A's My Organization > Client One entity maps to B's Client One entity. Computers are enabled. The entity names are illustrative; A and B use their own entity IDs.

STAGE	GLPI A	GLPI B
In your stock	DMOS-LT-002 Entity: My Organization	No copy. This example has no matching route yet.
After transfer	Same asset and same A ID. Entity: Client One	Work becomes eligible and is queued on A.
After verified sync	Same asset remains in A's Client One entity.	A linked copy exists in B's Client One entity.

Why the transfer starts sync

1. The entity changed. A checks the route again even if the Name and Comments did not change. The asset is now in the client's sync scope.

2. A queues the eligible work. The worker sends the approved target entity, Computer identity and enabled mapped values to B.

3. B resolves the asset safely. In this example, no B copy exists, so a complete unique no-match check permits a first copy in B's Client One entity. If a valid existing pair is found instead, reuse it and apply normal authority rules rather than creating or reseeding another asset.

4. A verifies and records the link. A keeps the original asset; B holds its linked representation. Sync does not delete the A record or physically move its database row into B.

> **Existing links need scope checks.** If an already-linked B asset is in the wrong entity for the new route, do not silently move it, relink it or create a duplicate. Report the scope mismatch for review under the supported transfer policy.

This is target behavior for a transfer into an enabled route, not proof that the new REST path supports creation or transfers today. If the organization entity also has a matching route, the asset may already have a B copy before the transfer.

Example: Return to Your Stock

DMOS-LT-002 is in A's **Client One** entity and already has a confirmed copy in Client One's GLPI B. The client returns the asset, so you transfer the original A record back to **My Organization**.

> **Assumption:** My Organization has no enabled route to this B connection. After the transfer, no other route to that connection matches the asset. These are target routing examples, not verified live transfer tests.

STAGE	GLPI A	GLPI B
With the client	DMOS-LT-002 Entity: Client One In scope for the client route.	Confirmed copy in Client One's entity. Normal mapped sync applies.
After return	Same asset and same A ID. Entity: My Organization	Existing copy stays in Client One's entity. No automatic deletion or move.
Later edits	Stock-side changes stay on A for this connection.	Old client copy remains untouched by further out-of-scope sync.

What stops, and what remains

1. A rechecks the route. The asset is no longer eligible for this client's connection. Stop sending new updates to that B copy, and do not apply further out-of-scope B values back to A.

2. Preserve the old B record. Do not delete, archive, clear or transfer it automatically. Keep the link and past sync evidence for auditing; being out of scope does not mean the historical identity should be forgotten.

3. Recheck queued work. Pending work must be checked against the current scope before another write. Already executed or in-flight work needs reconciliation, not deletion of its evidence. A transfer cannot undo a write that already completed.

> **Returning to the same client later:** verify and reuse the existing valid link. Resume normal field authority; do not create another B copy or repeat first-copy seeding on an existing asset.

Route matching is per B connection. If My Organization matches another enabled route, that connection may continue syncing. Returning to your organization is not a global stop switch.

Example: Client to Client

You transfer **DMOS-LT-003** from **Client One** to **Client Two** inside GLPI A. Its A asset ID remains the same. In this example, each client has a **different GLPI B installation**.

> **Routes:** A's Client One entity maps to Client One's B connection. A's Client Two entity maps to Client Two's B connection. Assume each route matches only its own client, and Client Two has no existing copy.

STAGE	GLPI A	CLIENT GLPI B INSTALLATIONS
Before transfer	DMOS-LT-003 Entity: Client One	Client One B: linked copy exists. Client Two B: no copy.
After transfer	Same asset and same A ID. Entity: Client Two	Client One B: leave copy untouched. Client Two B: new work becomes eligible.
After verified sync	Original asset remains in A's Client Two entity.	Client One B: old copy retained. Client Two B: copy created and linked after safe matching.

How the two connections behave

Old connection: once no route to Client One's B matches, stop updates in both directions for that connection. Retain the old B copy and its link for audit. Do not delete it or copy its client-specific values directly into Client Two's system.

New connection: evaluate Client Two's route, permissions and mappings independently. Reuse a verified existing link or unique permitted match if available. Create only after a complete no-match check, then confirm the link. First-copy seeding applies only to a genuinely new B asset; later updates follow Client Two's configured field authority.

If both clients use the same GLPI B

Do not treat an entity change as permission to create a second record for the same linked asset in that connection. The intended same-installation outcome is to reuse the B asset and move it to the newly routed B entity only through an explicitly supported, permission-checked GLPI transfer.

> **Current boundary:** automatic B-entity transfer is not qualified in the new REST path. If the existing B entity does not match the new route, hold the work and report a scope mismatch for review. Do not silently move, relink or duplicate the asset.

These examples explain target behavior and the current transfer limitation. They do not demonstrate completed creation, cross-client transfer or cleanup support in the new execution path.

Recovery and Readiness

10. Changes starting on B

The full design records relevant B-side changes in B's durable change queue. A checks for them even when nothing changed on A, applies the mapping rules, and sends any necessary corrections back to B.

A acknowledges the **exact B change records** it successfully handled. Until then, B retains them for recovery so a temporary network failure does not lose an edit.

In the new A-driven REST-records design, A sends requests to B and receives data in the responses. B does not need a separate callback to A for this exchange. The older Adapter mode communicates differently.

11. Failures, History and performance

Unavailable servers, stale work, permission failures and conflicts must remain visible. A timeout means the outcome may be unknown, not that nothing happened. Retained work IDs and execution journals help prevent retries from repeating writes or creating duplicate assets.

The target normal A-started batch uses **one POST and one GET**, with an optional acknowledgement request for returned B changes. Authentication, settings updates, polling, retries and additional pages are separate requests.

Batching and local B processing reduce network overhead, but do not eliminate CPU, RAM or database work. Sync is asynchronous, not guaranteed instant. Production capacity still needs measurement.

12. What works today

As of **21 October 2026**, the new path has verified foundations for authentication, bounded **Computer Name syncing**, batches, durable results, diagnostics and versioned settings.

Broader field support, reliable browser-change tracking for Both, B-change acknowledgements, creation, UUID/billing integration and large-scale qualification remain unfinished in this new path. Some capabilities exist in the older path, but must not be assumed complete in the replacement.

> **In one sentence:** A decides what should happen, B safely performs its local part, and A verifies and finishes the exchange, with both sides retaining enough evidence to recover when something goes wrong.

Implementation status is a dated snapshot, not a release or production-readiness guarantee.